Further Lean proof simplifications

`Computability/DiophantineConsistency`

Three concrete proposals, in two families, after treating all earlier proposals as applied.

Source revision	`3299f4aa4c9ae38b0f5a1c00a1f0d23f6cbdb114`
Toolchain	Lean 4.32.0; Mathlib revision `81a5d257c8e410db227a6665ed08f64fea08e997`
Exclusion baseline	41 research packets, a–zo, and the current `docs/incoming` retirement/index ledger
Evidence	Checked Lean kernels and literal source-table extractions; exact applicable proposal patches; complete integration builds still outstanding

Main finding. The best structural opportunity is to derive primitive recursiveness from the concrete scalar program that the project already maintains, eliminating an independent certification chain. The strongest local arithmetic opportunity is to consume only the source equation needed for a factor-two bound. The same positivity reasoning makes an external beta-sign assumption derivable from a positive source solution and permits unconditional positive-zero equivalences.

The report deliberately keeps the list short. Earlier packets already cover most broad automation, algebraic normalization, scanner, witness-layout, Pell, coefficient, carry, and controller abstractions. Repackaging those would not satisfy the requested exclusion.

This is a proposal and verification packet, not a claim that all production patches have passed a full repository build. Line reductions below compare with the pinned checkout; they are not savings measured after all previous proposals.

Contents

1 Results and the comparison standard **3**

1.1 What “assume already applied” means here . . . 3

1.2 Review coverage and limits . . . 3

2 S1: derive computability from the program already supplied **4**

2.1 The duplicated certification . . . 4

2.2 Break the import cycle before using the bridge . . . 4

2.3 Keep the recursively enumerable endpoint . . . 5

2.4 Why this survives the exclusion baseline . . . 5

2.5 Cost, preservation, and adoption . . . 5

3 B1: consume only the equation needed for the bound **5**

3.1 The current proof proves much more than its conclusion needs . . . 5

3.2 The short semantic client after packet b . . . 6

3.3 Two adoption choices . . . 6

3.4 Sharpness and boundaries . . . 6

4 B2: make the beta sign a consequence of a positive zero **7**

4.1 Both source presentations force the guard . . . 7

4.2 Remove the external parameter from the zero-set equivalence . . . 7

4.3 Preserve the distinction between solutions and arbitrary tuples . . . 8

4.4 Compatibility cost and a smaller direct implementation . . . 8

4.5 Why this is additional after prior work . . . 8

5 Verification evidence and what remains unverified **9**

5.1 Literal-table integrity . . . 9

6 Exclusions that materially shaped the result **10**

7 Recommended implementation order **10**

A Archive contents and reproduction **11**

B Sources and closest earlier proposals **12**

1 Results and the comparison standard

ID	Proposal	Additional contribution after earlier work	Current-pin accounting
S1	Use the existing realizer as the computability certificate	Remove the separate extensional primitive-recursion proof pipeline; change the dependency direction	608 to 392 lines across all affected/new modules
B1	Prove `source_Y_ge_two` from row 1	Remove row 0 and the beta-sign premise from the stronger local contract	Compatibility declaration: 21 to 13 lines
B2	Recover the beta guard from positive zeros	Strengthen gap/old zero-set equivalences to every port tuple	Additive compatibility patch: 90 new lines

S1 is the largest source-organization opportunity. B1 is the most contained proof edit. B2 simplifies theorem assumptions and downstream interfaces; the supplied compatibility-preserving implementation grows the source. B1 and B2 belong to the same positive-factor arithmetic family. The generic equivalence transport used in B2 is routine infrastructure, not a claimed fourth idea.

The reference checkout contains 702 Lean files and 87,259 physical Lean lines in the requested Lean subtree. The designated research directory contains 41 dated packets and 2,507 tracked files. At this revision, `docs/incoming` contains a README ledger, rather than a collection of loose incoming archives. The ledger still counts as prior work: its adopted, retired, uncompiled, optional, and future-work ideas all enter the exclusion set. These inventory facts are recorded in the accompanying audit artifacts and can be reproduced from the pinned Git tree.[1, 2, 3]

1.1 What “assume already applied” means here

There is no unique buildable checkout representing every historical proposal: some are alternative architectures, and some were deliberately left as research directions. Accordingly, this report applies the user’s instruction in two separate ways.

First, **novelty is evaluated against the proposed mechanisms**, including unadopted ones. A new spelling, target name, helper namespace, or client of an old generic mechanism is not counted as an additional idea. Second, **line counts use the real pinned source only**. They measure the supplied diffs, not marginal savings in an invented post-proposal repository. In particular, the 216-line S1 reduction and the eight-line B1 reduction must not be added to earlier packets’ promised savings.

The counterfactual contribution of S1 is the deletion of one independent proof obligation. The counterfactual contribution of B1 is a smaller mathematical dependency set. The contribution of B2 is a stronger endpoint without an external beta-sign parameter. These remain meaningful even after the earlier automation and shared-equation APIs are assumed available.

1.2 Review coverage and limits

The audit surveyed all principal report heading inventories, current packet READMEs, cumulative exclusion taxonomies, relevant detailed sections and technical notes, and candidate-specific searches of narrative and proposal artifacts. It also examined the incoming ledger. It does not claim a semantic rereading of every historical source snapshot, compiler log, or transitively linked external report. The archive contains a packet-by-packet ledger and explicit retain/reject decisions, so the scope of the absence evidence is inspectable.

Novelty here means *not found in the designated corpus under that review*, not a new result in recursion theory or arithmetic. The useful result is a precise change to this project’s proof dependencies and public contracts.

2 S1: derive computability from the program already supplied

Priority: highest structural payoff. Checked certificate bridge; production reorganization remains an integration proposal.

2.1 The duplicated certification

The current `Computability.lean` builds an extensional primitive-recursion derivation through register projections, arithmetic constructors, `pr_rawStep`, `pr_enumStep`, and `pr_boundedRun`. It ultimately proves `haltingTest_natPrimrec`, its vector counterpart, and recursive enumerability of the refutation-input predicate.[4]

Independently, `DirectScalarEnumerator.lean` supplies a concrete object:

```
def program : ScalarPRCode.Realizes haltingTest := ...
```

The code type already has `ScalarPRCode.primrec`; `Realizes f` is a code paired with pointwise correctness of its evaluation. Therefore a realizer itself supplies the extensional theorem:

```
theorem natPrimrec {f : ℕ → ℕ}
    (p : ScalarPRCode.Realizes f) : Nat.Primrec f :=
  p.val.primrec.of_eq p.property
```

This exact bridge was checked after freshly compiling a byte-identical copy of the pinned `ScalarPRCode.lean`. No project object file was substituted for that source compilation. Its axiom report contains only `Quot.sound`. The vector adapter was also checked, using the explicit `Mathlib.Computability.Primrec.List` import.[6, 5]

2.2 Break the import cycle before using the bridge

The existing direct enumerator imports `Computability`, because that module owns the semantics it realizes. Merely adding an import in the opposite direction would create a cycle. The proposal splits the semantic layer and reverses the certification dependency:

`EnumeratorSemantics` ⟶ `DirectScalarEnumerator` ⟶ `Computability`

Here an arrow means "is imported by." `Bridge` continues to import `Computability`. The separate direct-arithmetic branch still provides `ScalarPRCode`; it does not need the extensional computability chain.

Module	Responsibility after the split
`EnumeratorSemantics`	Unchanged definitions of `boundedRun`, `initialState`, `refutationInputs`, and `haltingTest`; iteration/halting semantic lemmas; private encoding and decidability infrastructure required by the existing definition.
`DirectScalarEnumerator`	Existing concrete program and realization proofs. Only its semantics import changes.
`Computability`	Derive primitive recursiveness from the supplied realizer; retain the vector conversion; derive recursive enumerability from search for the scalar test value 1.

The private encoding instance needs attention. The old `haltingTest` uses a decidable equality for the bounded state. Moving its declaration while accidentally changing instance selection could change the elaborated definition even if the displayed term appears similar. The candidate moves the existing `stateEquiv`, `Primcodable St`, and `DecidableEq St` infrastructure with the semantics. The report claims source-body preservation, not byte identity of elaborated constants with renamed private dependencies.

2.3 Keep the recursively enumerable endpoint

The search predicate can be expressed using the same certified scalar function. Primitive recursiveness gives computability of

$$(u, t) \longmapsto \text{decide}(\text{haltingTest}(t) = 1).$$

The unused first component supplies the input to the partial search. Apply `Partrec.rfind`, then `dom_re`, and use the retained semantic equivalence

$$(\exists t,\ \text{haltingTest}(t) = 1) \ \longleftrightarrow\ C222\ F.$$

The proposed full `Computability.lean` is included, so the suggestion does not leave the old recursive-enumerability endpoint unexplained. The complete generic search-to-RE adapter was independently compiled with the scalar function and its semantic equivalence as explicit parameters; its axiom report contains only the three standard axioms listed below. This checks the search proof, while the concrete program binding and production module integration remain separate obligations.[4, 10]

This is an architectural choice with a tradeoff. The new path relies on the explicit program and its realization proof. Keeping the old extensional proof as an independent regression derivation could be valuable, but then it should live in an optional audit module and should not be counted as deleted from the maintained certification pipeline. The proposal does not claim a runtime or elaboration speedup; importing the direct program can make this module's dependency closure more expensive.

2.4 Why this survives the exclusion baseline

Packet i already proposes `fun_prop` automation of the constructor-composition proofs in this very module, while keeping supplied program construction and realization proofs explicit. Packet e proposes sharing graph-enumerator frontends; packet r proposes sharing rule schedules. All are assumed applied.[15, 13, 16]

Even after those changes, an automated extensional proof chain and a concrete realizer can remain two independently maintained certifications. S1 makes the latter discharge the former's endpoint. The retained novelty is that dependency inversion and proof elimination, not primitive-recursion automation, scanner fusion, or a new frontend DSL.

2.5 Cost, preservation, and adoption

The original two affected modules have 608 physical lines. The full candidate, including the new semantic module, has 392: a 216-line reduction relative to the pin. `DirectScalarEnumerator` is unchanged after its imports. Public semantic definitions keep their names and source bodies; the public theorem endpoints remain available from `Computability`.

The supplied patch passes `git apply --check`. The generic certificate, vector bridge, and search-to-RE adapter pass Lean. An independent static review followed 19 project modules in the affected import closure and found no project cycle or unresolved project import; Mathlib and Lean remain explicit boundaries of that static check. Sparse-missing project sources were fetched at the exact pin and verified against their Git blob identifiers. The complete module split, the concrete production endpoint, and downstream integration have not been established by a full project build. Before adoption, build the three changed/new modules and `Bridge`, then replay the existing named-endpoint axiom checks. Inspect executable program data independently of proof irrelevance. Do not turn a source-level preservation check into a claim about generated machine code.

3 B1: consume only the equation needed for the bound

Priority: smallest local edit. Arithmetic, production candidate body, and literal-table binding checked.

3.1 The current proof proves much more than its conclusion needs

The target is `NativePellBootstrap.source_Y_ge_two`, at lines 95–115 of the pin. Write

$$\beta = \text{ports}(0), \quad J = x_0, \quad L = x_{24}, \quad N = x_{16}, \quad s = x_9, \quad q = 1 + \beta J.$$

The source coordinates are natural numbers, viewed in the integers inside the polynomial equations. The existing proof obtains

$$q \geq 2, \qquad L = q^2 - 1 \geq 3, \qquad N = q(1 + L) \geq 8,$$

and then concludes only $Ns \geq 2$. It uses rows 0 and 1 and an external $\beta > 0$ premise.[7]

The row-1 equation alone is

$$-1 - L + N - \beta J - \beta JL = 0,$$

or equivalently

$$N = (1 + \beta J)(1 + L) = q(1 + L).$$

If $N, L, s > 0$, then $1 + L > 0$ and $N > 0$ force $q > 0$. Since q is an integer, $q \geq 1$. Positivity of the natural L gives $1 + L \geq 2$, so $N \geq 2$. Finally $s \geq 1$ yields $Ns \geq N \geq 2$.

Neither row 0, the cubic identity, positivity of J, nor a sign premise on β is needed for this minimal contract. This is a strictly smaller set of assumptions than the original theorem uses.

3.2 The short semantic client after packet b

Packet b already proposes a common interface for the source-register equations and explicitly names `source_Y_ge_two` as a client. Its decoding, unfolding, and cast savings belong to the assumed baseline.[11]

After that interface, the additional client can be as small as the following checked lemma:

```
theorem semantic_lower {q L N s : ℕ}
    (hq : 1 ≤ q) (hL : 0 < L) (hs : 0 < s)
    (hN : N = q * (1 + L)) : 2 ≤ N * s := by
  have hN2 : 2 ≤ N := by rw [hN]; nlinarith
  exact hN2.trans (Nat.le_mul_of_pos_right N hs)
```

The archive also checks the stronger signed version, deriving $q > 0$ from $N > 0$ rather than requiring it. It binds the proof to the exact maintained 22-row table, not merely to an independently simplified polynomial.

3.3 Two adoption choices

A contained proof-only edit. Keep the old theorem header and use the nonnegative consequence of $\beta > 0$ directly on row 1. The supplied patch replaces a 21-line declaration by a 13-line declaration, introduces no helper or import, and retains the original public type. The exact candidate body is checked in the extracted-table prototype.

A stronger reusable endpoint. Expose a theorem using only the three positive coordinates and row 1, with no beta premise. Keep the old declaration as a wrapper if callers need its signature. Count the helper and wrapper lines when evaluating source savings. Propagate the stronger endpoint only where the beta argument's remaining uses have been examined.

The first option has a small, concrete raw-source reduction. The second has a cleaner logical interface and can simplify consumers such as `source_rank_half_lt` and `source_base_separation`. It does not automatically imply a net line reduction once compatibility is included.

3.4 Sharpness and boundaries

The factor-two target is sharp for the isolated row-1 contract: $\beta = 0$, $J = 0$, $L = 1$, $N = 2$, and $s = 1$ satisfy it and give $Ns = 2$. This is *not* a complete positive source solution, and it does not satisfy row 0. It only demonstrates why the smaller theorem should not demand $N \geq 8$ or strict beta positivity.

Positivity assumptions matter. If $L = 0$ is allowed, $q = N = s = 1$ satisfies $N = q(1+L)$ but gives $Ns = 1$. If $s = 0$, the product bound fails. The signed no-beta proof uses $N > 0$ to determine the sign of q; dropping it allows $q = N = 0$. These are local-contract counterexamples, not claims about the full Diophantine system.

The useful simplification is mathematical support reduction. Replacing one tactic by another, or extracting a repeated row identity that packet b already supplies, is not the new idea.

4 B2: make the beta sign a consequence of a positive zero

Priority: stronger public interfaces. Checked new reasoning; additive integration patch retains all existing APIs.

4.1 Both source presentations force the guard

The old and gap source systems share row 0. Their first two rows normalize to

	row 0	row 1
old	$L = q^2 - 1$	$N_{\text{old}} = q(1 + L)$
gap	$L = q^2 - 1$	$N_{\text{gap}} = qL$

$$q = 1 + \beta J.$$

For positive source coordinates, both right-hand cofactors are positive. Thus row 1 forces $q > 0$. The positive integer q cannot equal 1, because row 0 would give $L = 0$. Consequently $q \geq 2$, so $\beta J = q - 1 > 0$. Since $J \geq 0$, it follows that $\beta > 0$.[8, 9]

This argument is local. It uses polynomial equations and order; it invokes no global consistency, existence, Pell classification, or final theorem. For natural-valued coordinates, nonnegativity of J is automatic; the scalar theorem does not need a separately supplied $J > 0$ premise.

The checked common kernel takes $D = L$ or $D = 1 + L$:

```
theorem beta_pos_kernel {β J L N D : ℤ}
    (hJ : 0 ≤ J) (hL : 0 < L)
    (hN : 0 < N) (hD : 0 < D)
    (hquad : L = (1 + β * J)^2 - 1)
    (hlinear : N = (1 + β * J) * D) : 0 < β := ...
```

The full proof and both literal-row wrappers are in `GuardDischarge.lean`. The source tables were copied exactly into separate namespaces and checked against the production declarations by byte comparison.

4.2 Remove the external parameter from the zero-set equivalence

Currently `GapSource66.positiveZeroEquiv` accepts $h_\beta : 0 < \text{ports}(0)$. Its domain and codomain are subtypes of positive common zeros. Each element of either subtype already supplies a proof of this guard. Hence the equivalence can be provided for *every* port tuple.

Let A and B be the two positive-zero subtypes and G the proposition $\beta > 0$. The existing theorem supplies $e : G \to (A \simeq B)$. The new arithmetic supplies $g_A : A \to G$ and $g_B : B \to G$. The following complete constructor was checked:

```
def discharge {A B : Type*} {G : Prop}
    (e : G → A ≃ B) (ga : A → G) (gb : B → G) : A ≃ B where
  toFun a := e (ga a) a
  invFun b := (e (gb b)).symm b
  left_inv a := (e (ga a)).left_inv a
  right_inv b := (e (gb b)).right_inv b
```

Proof irrelevance aligns the potentially different proofs of the proposition G in the inverse equations. No global inhabitant of either solution subtype is required: if $\beta \leq 0$, both are empty, and the per-element construction remains well-defined.

The proposed additive API is:

```
gap_positive_beta
old_positive_beta
positiveZeroEquiv_all
exists_positive_zero_iff_all
exists_positive_nat_zero_iff_all
```

The natural-number existence theorem reuses the existing positive-integer/natural transport. That transport and the generic equivalence constructor are not counted as new research mechanisms. The new endpoint is the unconditional positive-zero equivalence enabled by the recovered port sign.

4.3 Preserve the distinction between solutions and arbitrary tuples

The external beta guard cannot simply be erased from every positivity theorem. `toOld_positive`, for example, currently acts on an arbitrary positive tuple without source equations. Take all coordinates equal to 1 and $\beta = -3$. Then $q = -2$ and the transformed coordinate 16 is $1 + q = -1$. Unguarded positivity transport at that generality is false.

Row 0 alone also does not establish the sign: $J = 1$, $\beta = -3$, and $L = 3$ give $q = -2$ and $L = q^2 - 1$. Row 1 with a positive output is what excludes the negative root. Nonnegativity of J is likewise substantive: allowing $J = -1$ and $\beta = -1$ gives $q = 2$, $L = 3$, and positive old output $N = 8$.

The argument proves only $\beta > 0$. It does not supply larger calibration assumptions such as $\beta \geq 15$. The local first-two-row system admits $J = 1$, $\beta = 1$, $L = 3$, and old $N = 8$, so those two equations cannot justify such a stronger bound. Existing quantitative hypotheses must be reviewed independently.

4.4 Compatibility cost and a smaller direct implementation

The supplied integration is deliberately additive: one shared helper file, one import, and appended guarded-to-unguarded adapters. It retains all existing declarations and adds 90 lines. Its value is a simpler public contract and easier downstream premise management. It is not offered as an immediate source-size win.

If the stronger endpoints prove useful, a later direct refactor can avoid recovering β where only positivity of the affine maps is needed. For a gap zero, $N_{\mathrm{gap}} = qL > 0$ already gives $q > 0$, which suffices for the forward map's changed coordinate. For an old zero,

$$N_{\mathrm{old}} - q = qL > 0$$

follows directly from row 1 and positivity, without using $N = q^3$ or row 0. The other changed gap coordinate uses the existing independent rows 3, 5, and 6. This is an alternative implementation of the same family, not another counted proposal, and no unimplemented line saving is assigned to it.

4.5 Why this is additional after prior work

Packet b supplies the normalized source-register identities; it does not recover the external beta sign from membership in the positive zero set. Packet f's `PortBounds.beta_pos` projects a stronger assumed port bound. Packet zj's positivity improvement derives positivity of a selected source digit in a different construction. None supplies the particular solution-dependent port guard needed here.[11, 14, 17]

The closest prior is acknowledged rather than displaced. B2 is a second application of the same arithmetic viewpoint as B1, with a different useful public endpoint.

5 Verification evidence and what remains unverified

Artifact	What was checked	Result
`RealizerPrimrecBridge.lean`	Exact pinned scalar-code source freshly compiled; natural and vector certificate bridges	Lean exit 0
`GenericREAdapter.lean`	Full search-to-recursive-enumerability proof with function and semantic equivalence supplied as parameters	Lean exit 0
`MinimalPellBootstrap.lean`	Scalar bounds, stronger contracts, beta recovery, exact source table, and compatibility candidate body; ten named axiom reports	Lean exit 0
`GuardDischarge.lean`	Both exact row tables; sign recovery; generic equivalence; existence and nonpositive-beta emptiness adapters	Lean exit 0
Three unified diffs	Context applicability against the pinned source; no mutation required	All checks pass
S1 static import review	Nineteen project modules; exact import-only change to the direct enumerator; no project cycle	Pass, static only
Full production modules	Actual imports, all old and new declarations together, downstream project clients	Not rebuilt

The compiler is Lean 4.32.0, commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`. The Mathlib source revision matches the project's pinned `81a5d257c8e410db227a6665ed08f64fea08e997`. The tests use existing Mathlib dependency artifacts; the project source copied for S1 was freshly compiled in the proposal workspace. No stale project object was used to claim that the production patches had passed.

The successful named checks report only combinations of `propext`, `Classical.choice`, and `Quot.sound`; none reports `sorryAx` or a new user axiom. The initial vector-bridge attempt lacked the `Primrec.List` import and failed. Its log is retained alongside the corrected successful run. A corrected import is part of the actual evidence, not a reason to hide the first attempt.

The B2 extraction supplies the *existing guarded equivalence* as an explicit parameter. Its result therefore checks the new arithmetic and transport, not a fresh proof of all existing affine identities. This matches the user's instruction to assume earlier work available while keeping the verification boundary visible.

5.1 Literal-table integrity

The canonical extraction includes the final line feed after the table's closing bracket:

Declaration	Bytes	SHA-256
Old `sourceRows`	1,774	`bb4b3bba67a6c924be4659789ddad7e87fa7cb050f1343dc939e6e74a523deca`
Gap `rows`	1,609	`5f7a9e6ee3111fd9aa3debd60c925d81ec731c58a58a7df0817cf6f0d49d3387`

The old table is byte-identical in the pinned source, the algebra prototype, and the guard prototype. The metadata also records the initial no-final-LF hash convention, explaining the different digest in any earlier local receipt. The function body did not change.

Patch applicability is weaker than Lean elaboration. A checked extracted body is stronger than an unchecked sketch, but weaker than rebuilding its production module with the project's complete import closure. These distinctions govern every status label and saving claim in this report.

6 Exclusions that materially shaped the result

The audit applies the exclusion to mechanisms, not only to adopted patches. Representative rejected candidates are listed here; the complete packet ledger is in `audit/novelty_audit.md`.

Candidate considered	Why it was excluded
Elementary polynomial row operations and invertible transformations	Packet c explicitly covers row-addition zero-set equivalence and the later invertible linear-combination extension. A new triangular system is another client.
Finite radix-prefix bounds from a digit-length theorem	Packet zo explicitly rejects this generic prefix/digit-length mechanism as already covered; packet i already uses the library inequality.
Aligned cells as a subgroup, additive equivalence, or group action	Repeatedly proposed or excluded since packet b, including later i/k/r/u audits.
Reduce the whole copy kernel modulo two	Packet zj explicitly records this as prior scalar-projection/carry-parity work.
Further scanner, syntax-codec, witness-layout, coefficient-support, Pell-error, or controller-order frameworks	Covered by the ten-family cumulative taxonomy, including optional and future-work variants.

A possible halted-run saturation helper was also investigated. It had only a few small current callers, so helper cost was likely to exceed removed case splits. It was not retained merely because no exact earlier narrative match was found. The report requires a useful concrete target as well as a plausible novelty boundary.

The most important near-overlap is B1: the earlier packet names exactly the same theorem. The report therefore claims only the smaller assumptions and omitted quadratic dependency, never the already-proposed shared decoding. The corresponding near-overlap for S1 is `fun_prop`; it automates a chain that S1 proposes to make unnecessary.

7 Recommended implementation order

1. Adopt or reject B1 locally. Its compatibility patch is small and its exact body has an extracted-table check. Rebuild `NativePellBootstrap` and its immediate clients. Keep the stronger no-beta theorem only if its call sites make the additional API worthwhile. Compare complete declarations, helpers, imports, and wrappers rather than isolated proof bodies.

2. Evaluate S1 as a module-boundary change. Move semantic definitions first, preserve their encoding/decidability choices, then redirect the direct enumerator's import. Derive the extensional endpoints from the realizer. Build the new module order and the downstream bridge. Decide explicitly whether an independent extensional derivation belongs in an optional audit target; retaining it changes the savings calculation.

3. Introduce B2 where it removes repeated port premises. The additive APIs allow gradual use without rewriting established callers. Start at positive-zero equivalences and source-only consumers. Retain external guards for arbitrary tuples and for independent quantitative bounds. If many clients migrate, a direct positivity-based implementation can later replace the compatibility layer; count that refactor separately.

No package-wide automation, wholesale signature rewrite, or broad search-and-replace is required for these proposals. The necessary next verification is a focused production build of the affected dependency closures, followed by the existing endpoint audits.

A Archive contents and reproduction

Path	Purpose
`report.tex`, `report.pdf`	This review, editable source and rendered document.
`lean/`	Standalone checked prototypes, exact scalar-code source, pinned toolchain, small Lake configuration, and reproduction script.
`patches/`	Three exact unified diffs against the source pin.
`proposed/`	Complete candidate production files for structural and guard changes.
`audit/`	Novelty review, comparison decisions, source inventory, candidate findings, and integrity metadata.
`evidence/`	Compiler logs, scoped receipts, and patch-applicability results.
`scripts/`	Integrity verification and report build helpers.
`SHA256SUMS`	Digests of delivered files; excludes itself.

The README gives exact commands. In an environment with the pinned toolchain and dependencies available, the standalone examples can be replayed through the small Lake project. Acquiring Mathlib artifacts may require a network-enabled environment. The reproduction helper checks focused examples only; it does not silently apply production patches or claim a complete repository build.

For production review, use a clean checkout at the recorded revision, run `git apply --check` on the desired patch, apply it in a review branch, and build the named changed modules and clients. The supplied pin, literal-table hashes, and source inventories make a later comparison meaningful if `main` moves.

B Sources and closest earlier proposals

All project links below resolve to the same pinned revision. The short descriptions identify the exact evidence used; a link to a previous report is an exclusion reference, not an endorsement of its unverified savings.

References

[1] Vladimir Reshetnikov, ProveIt. Source commit `3299f4aa4c9a`, 26 October 2026.

[2] Research packet directory, 41 packets a–zo, including README and ADOPTION ledgers.

[3] `docs/incoming/README.md`, incoming/retirement index and earlier-work summaries.

[4] `Computability.lean`, particularly private constructor pipeline and lines 243–334: bounded semantics, recursive enumerability, and primitive-recursion endpoints.

[5] `DirectScalarEnumerator.lean`, imports and lines 267–270: supplied `program` realizing `haltingTest`.

[6] `ScalarPRCode.lean`, lines 36–47: `primrec` and `Realizes`.

[7] `NativePellBootstrap.lean`, lines 95–138 and 348–363: factor-two bound and direct consumers.

[8] `SplitCertificate59.lean`, literal `sourceRows` table.

[9] `GapSource66.lean`, literal gap rows and lines 239–348: positivity, affine equivalence, and natural/integer existence.

[10] Mathlib, revision `81a5d257c8e410db227a6665ed08f64fea08e997`. `Computability/RE.lean` and primitive-recursion support modules.

[11] Packet b, `report.tex`, lines 617–641: shared source-register equations; explicitly includes `source_Y_ge_two`. Nearby aligned-cell proposals also enter the exclusion.

[12] Packet c, `report.tex`, lines 774–808: elementary row operations and invertible row-combination extension.

[13] Packet e, `report/report.tex`, section 14: shared graph-enumerator frontend, with distinct realizers/costs.

[14] Packet f, architecture section, lines 86–97, and accompanying port-bound APIs: beta positivity as a projection of stronger external assumptions.

[15] Packet i, `report.tex`, lines 313–346: extensional primitive-recursion automation while leaving code construction and realization explicit.

[16] Packet r, `report.tex`, shared numeric/program rule schedules.

[17] Packet zj: selected-digit source positivity; `evidence/current_controller_findings.md` excludes whole-kernel parity projection.

[18] Packet zo audit, candidate C9: generic finite-prefix/digit-length bounds already excluded.